SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 40%

Duration: 1 Hour

QUESTIONS: 3 **Total Marks:30**

Annexure A

Dry Run Evaluation

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

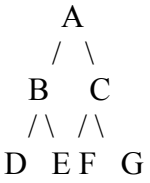
Signature of the student:

Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph:



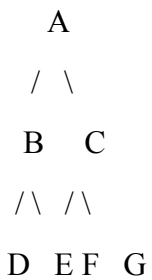
Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

Iteration	Queue	Visited Node
1		
2		
3		
4		
5		
6		
7		

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. **Initial State :**
   1 3 6
   5 0 2
   4 7 8

Goal State :

1 2 3
4 5 6
7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.
Fill in the table.

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1			
2			
3			
4			
5			

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	**20**	Clearly understands the search problem, initial state, goal state, and search strategy.	Understands the problem with minor misunderstandings.	Demonstrates partial understanding of the search problem.	Shows little or no understanding of the search problem.

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
State Expansion and Dry Run Execution	**30**	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	**20**	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	**20**	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	**10**	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

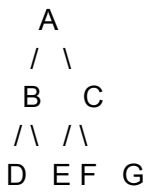
1. Breadth-First Search (BFS) Using the Given Graph

Problem Statement

Breadth-First Search (BFS) is an uninformed search algorithm that explores a graph level by level. It begins at the starting node and visits all its immediate neighbours before moving to the next level.

BFS uses a FIFO (First In, First Out) queue. The node inserted first is expanded first. A visited list is maintained to avoid processing the same node repeatedly.

Given Graph:



Starting Node: A

Step 1: Apply BFS

Initially, the queue contains only the starting node A.

Queue = [A]

The algorithm removes the front node, visits it, and adds its unvisited children to the rear of the queue.

The nodes are explored from left to right.

Step 2: Complete the BFS Dry-Run Table

Iteration	Queue Before Expansion	Visited Node	Queue After Expansion
1	[A]	A	[B, C]
2	[B, C]	B	[C, D, E]
3	[C, D, E]	C	[D, E, F, G]
4	[D, E, F, G]	D	[E, F, G]

5	[E, F, G]	E	[F, G]
6	[F, G]	F	[G]
7	[G]	G	[]

Note: Queue contents show the nodes waiting to be expanded. Children are inserted from left to right.

Question 1: Write the BFS Traversal Order.

The BFS traversal order is:

A → B → C → D → E → F → G

Explanation:

1. A is visited first because it is the starting node.
2. B and C are visited next because they are the immediate children of A.
3. D and E are visited after B.
4. F and G are visited after C.

Therefore, BFS visits all nodes at one level before proceeding to the next level.

Question 2: Show the Queue Contents After Each Iteration.

The queue contents after each iteration are:

Iteration 1: [B, C]

Iteration 2: [C, D, E]

Iteration 3: [D, E, F, G]

Iteration 4: [E, F, G]

Iteration 5: [F, G]

Iteration 6: [G]

Iteration 7: []

The queue becomes empty after all seven nodes have been visited.

Result

Breadth-First Search successfully traverses the given graph in the order A → B → C → D → E → F → G. The FIFO queue ensures that nodes are explored level by level, making BFS suitable for finding the shortest path in an unweighted graph.

2. Depth-First Search (DFS) Using the Same Graph

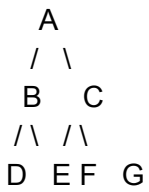
Problem Statement

Depth-First Search (DFS) is an uninformed search algorithm that explores one branch of a graph as deeply as possible before backtracking to explore another branch.

DFS can be implemented using a stack that follows the LIFO (Last In, First Out) principle or through recursion.

For the given graph, DFS begins at node A and explores the left branch before the right branch.

Given Graph:



Starting Node: A

Step 1: Apply DFS

Initially, the stack contains only node A.

Stack = [A]

For this dry run, the top of the stack is shown on the left. When expanding a node, its right child is pushed first, followed by its left child. This ensures that the left child is visited first.

Step 2: Complete the DFS Dry-Run Table

Iteration	Stack Before Expansion	Visited Node	Stack After Expansion
1	[A]	A	[B, C]
2	[B, C]	B	[D, E, C]
3	[D, E, C]	D	[E, C]
4	[E, C]	E	[C]
5	[C]	C	[F, G]

6	[F, G]	F	[G]
7	[G]	G	[]

Note: The stack is displayed with its top on the left. The children of each node are pushed in right-to-left order so that the left child is processed first.

Question 1: Write the DFS Traversal Order.

The DFS traversal order is:

A → B → D → E → C → F → G

Explanation:

1. DFS starts at A.
2. It moves to B and then explores D.
3. Since D has no children, DFS backtracks to B and visits E.
4. After completing the left subtree, DFS returns to A and visits C.
5. It then explores F followed by G.

Therefore, DFS explores one branch completely before moving to another branch.

Question 2: Show the Stack Contents After Each Iteration.

The stack contents after each iteration are:

Iteration 1: [B, C]

Iteration 2: [D, E, C]

Iteration 3: [E, C]

Iteration 4: [C]

Iteration 5: [F, G]

Iteration 6: [G]

Iteration 7: []

The stack becomes empty after all seven nodes have been visited.

Comparison Between BFS and DFS

Factor	**BFS**	**DFS**

Data structure	FIFO queue	LIFO stack or recursion
Exploration method	Level by level	Depth-wise
Traversal order	A, B, C, D, E, F, G	A, B, D, E, C, F, G
Memory requirement	Can be high for wide graphs	Often lower for wide graphs
Shortest-path guarantee	Yes, for unweighted graphs	No
Backtracking	Not required in the same way as DFS	Used when a branch ends

Result

Depth-First Search successfully traverses the given graph in the order A → B → D → E → C → F → G. The stack enables DFS to explore the deepest available branch first and backtrack when a node has no unvisited children.

3. Solving the 8-Puzzle Using Breadth-First Search (BFS)

Problem Statement

The 8-puzzle is a search problem consisting of a 3 × 3 board containing eight numbered tiles and one empty space represented by 0.

The objective is to transform the initial state into the goal state by moving the empty space up, down, left, or right whenever the move is valid.

Initial State:

1	**3**	**6**
5	0	2
4	7	8

Goal State:

1	**2**	**3**
4	5	6
7	8	0

BFS is used to explore the possible states systematically until the goal state is reached.

Step 1: Define the State Representation

Each arrangement of the eight numbered tiles and the empty space represents a state.

The initial state is:

1 3 6
5 0 2
4 7 8

The goal state is:

1 2 3
4 5 6
7 8 0

The empty space, represented by 0, can move only when the destination position is within the boundaries of the 3 × 3 board.

For this dry run, the possible moves are considered in the following order:

Up → Down → Left → Right

BFS uses a FIFO queue to store the states waiting to be expanded. A visited set prevents the algorithm from generating and enqueuing duplicate states.

Step 2: Trace the First Five BFS Iterations

The question provides a table with five iterations. However, the goal state is not reached within these five expansions. Therefore, the following table shows the first five iterations, followed by the complete solution path.

For readability, each board is also represented as a nine-digit sequence, reading from left to right and top to bottom. The digit 0 represents the empty space.

For example:

136502478 represents:

1 3 6
5 0 2
4 7 8

Iteration 1

Current State:

1	**3**	**6**
5	0	2
4	7	8

Queue Before Expansion:

[136502478]

The empty space is in the centre, so it can move up, down, left, or right.

The resulting states are:

1. Up: 106532478
2. Down: 136572408
3. Left: 136052478
4. Right: 136520478

Queue After Expansion:

[106532478, 136572408, 136052478, 136520478]

Iteration 2

Current State:

1 0 6

5 3 2

4 7 8

Queue Before Expansion:

[106532478, 136572408, 136052478, 136520478]

The empty space can move left, right, or down. The state obtained by moving down is the initial state and has already been visited, so it is not added again.

New states:

1. Left: 016532478
2. Right: 160532478

Queue After Expansion:

[136572408, 136052478, 136520478, 016532478, 160532478]

Iteration 3

Current State:

1 3 6

5 7 2

4 0 8

Queue Before Expansion:

[136572408, 136052478, 136520478, 016532478, 160532478]

The empty space can move up, left, or right. Moving up returns to an already visited state.

New states:

1. Left: 136572048
2. Right: 136572480

Queue After Expansion:

[136052478, 136520478, 016532478, 160532478, 136572048, 136572480]

Iteration 4

Current State:

1 3 6

0 5 2

4 7 8

Queue Before Expansion:

[136052478, 136520478, 016532478, 160532478, 136572048, 136572480]

The empty space can move up, down, or right. Moving right returns to an already visited state.

New states:

1. Up: 036152478
2. Down: 136452078

Queue After Expansion:

[136520478, 016532478, 160532478, 136572048, 136572480, 036152478, 136452078]

Iteration 5

Current State:

1 3 6

5 2 0

4 7 8

Queue Before Expansion:

[136520478, 016532478, 160532478, 136572048, 136572480, 036152478, 136452078]

The empty space can move up, left, or down. The state produced by moving left has already been visited.

New states:

1. Up: 130526478
2. Down: 136528470

Queue After Expansion:

[016532478, 160532478, 136572048, 136572480, 036152478, 136452078, 130526478, 136528470]

Summary of the First Five Iterations

Iteration	Current State	Queue Before Expansion	Queue After Expansion
1	136502478	[136502478]	[106532478, 136572408, 136052478, 136520478]
2	106532478	[106532478, 136572408, 136052478, 136520478]	[136572408, 136052478, 136520478, 016532478, 160532478]
3	136572408	[136572408, 136052478, 136520478, 016532478, 160532478]	[136052478, 136520478, 016532478, 160532478, 136572048, 136572480]
4	136052478	[136052478, 136520478, 016532478, 160532478, 136572048, 136572480]	[136520478, 016532478, 160532478, 136572048, 136572480, 036152478, 136452078]
5	136520478	[136520478, 016532478, 160532478, 136572048, 136572480, 036152478, 136452078]	[016532478, 160532478, 136572048, 136572480, 036152478, 136452078, 130526478, 136528470]

Step 3: Determine the Complete Solution Path

BFS continues expanding the remaining states in FIFO order until it reaches the goal state.

For the given initial and goal states, a shortest solution requires eight moves of the empty space.

The solution path is:

Initial State:

1 3 6

5 0 2

4 7 8

Move 1: Right

1 3 6

5 2 0

4 7 8

Move 2: Up

1 3 0

5 2 6

4 7 8

Move 3: Left

1 0 3

5 2 6

4 7 8

Move 4: Down

1 2 3

5 0 6

4 7 8

Move 5: Left

1 2 3

0 5 6

4 7 8

Move 6: Down

1 2 3

4 5 6

0 7 8

Move 7: Right

1 2 3

4 5 6

7 0 8

Move 8: Right

1 2 3

4 5 6

7 8 0

The goal state has been reached.

Therefore:

Total Moves = 8

Solution Path:

Initial → Right → Up → Left → Down → Left → Down → Right → Right → Goal

Step 4: Number of States Generated

The number of states generated depends on the order in which successor states are examined and when the goal test is performed.

For a reproducible count, the following conventions are used:

- Successor order: Up, Down, Left, Right.

- Duplicate states are not added to the queue.
- The initial state is counted as generated.
- The goal test is performed when a state is removed from the queue for expansion.

Under these conventions, the goal state is reached at the 311th expansion, after 500 unique states have been discovered, including the initial state.

Therefore:

Total Unique States Generated = 500

Total States Expanded Until the Goal Is Reached = 311

The number of states generated is different from the number of states expanded because some generated states remain in the queue when the goal is reached.

Note: The question's five-row table is insufficient to show the complete search. The five iterations above are the beginning of the dry run, while the solution path shows the complete shortest route.

Question 1: Which Node Is Expanded in Each Iteration?

During the first five iterations, the expanded states are:

Iteration 1: 136502478

Iteration 2: 106532478

Iteration 3: 136572408

Iteration 4: 136052478

Iteration 5: 136520478

The algorithm continues expanding the oldest state in the queue until the goal state is removed from the queue.

Question 2: How Many States Are Generated in Total?

Using the stated successor order and counting conventions:

Total unique states generated = 500.

The goal is reached at the 311th expansion.

Question 3: Write the Complete Solution Path from the Initial State to the Goal State.

The sequence of moves is:

Right → Up → Left → Down → Left → Down → Right → Right

The complete path contains eight moves and reaches the required goal state.

Question 4: Why Does BFS Always Find the Shortest Solution in an Unweighted 8-Puzzle Problem?

BFS explores states in increasing order of their depth from the initial state. All legal moves in the 8-puzzle have the same cost of one move.

Therefore, every state at depth 1 is explored before states at depth 2, every state at depth 2 is explored before states at depth 3, and so on.

When BFS first reaches the goal state, it has found a solution with the minimum possible number of moves.

For example, if a goal state can be reached in eight moves, BFS will explore all reachable states at smaller depths before accepting that solution.

Thus, BFS guarantees a shortest solution in an unweighted state space, provided that the search is performed correctly and the goal is tested in the standard FIFO order.

However, BFS may require substantial memory because it stores many generated states in its queue.

Result

Breadth-First Search can solve the given 8-puzzle by systematically exploring states in increasing order of depth. The shortest solution requires eight moves of the empty space:

Right → Up → Left → Down → Left → Down → Right → Right.

The goal state is reached after 311 expansions under the stated successor order and queue-processing convention, with 500 unique states discovered. BFS guarantees the shortest solution because every move has equal cost and states are expanded level by level.